# Rapport du Projet PRIM

Mouhamed Nourou Dine Cisse

January 4, 2025

## 1 Introduction

Le projet s'inscrit dans un contexte de développement d'un programme capable de lire, traiter et écrire des images aux formats PGM (niveaux de gris) et PPM (images couleurs). L'objectif principal est d'appliquer divers traitements aux images, tels que :

- Lecture et écriture de fichiers PGM et PPM.
- Transformations des images par modification des pixels, comme la conversion, l'éclaircissement, l'inversion, etc.

Ce rapport présente les choix techniques, la méthodologie adoptée, les problèmes rencontrés et les solutions mises en œuvre pour réaliser ce projet.

## 2 Choix et Méthodologie

### 2.1 Structures de Données

Les images sont représentées par une structure nommée `picture`, contenant :

- `width`, `height` : dimensions de l'image.
- `channels` : 1 pour PGM (niveaux de gris), 3 pour PPM (couleurs).
- `data` : tableau unidimensionnel contenant les pixels.

**Remarque :** La structure `picture` est déclarée dans `pictures.h` pour éviter une déclaration implicite et faciliter la modularité du code.

### 2.2 Modules Développés

Le projet est structuré en plusieurs modules pour une meilleure organisation et réutilisabilité du code :

- **pictures.c / pictures.h** : création du type `picture` et manipulation des images (lecture, écriture, manipulation).

- **pixels.c / pixels.h** : manipulation directe des pixels (modification des valeurs, etc.).
- **lut.c / lut.h** : gestion des transformations via les LUT (Look-Up Tables) pour des opérations rapides sur les pixels.
- **apply_filter.c / apply_filter.h** : application des transformations sur les images via une fonction `wrapper` dans `main.c`.
- **filename.c / filename.h** : gestion des chemins et des noms de fichiers pour simplifier la génération des noms de fichiers de sortie.

### 2.3 Utilisation d'un Makefile

Un `Makefile` a été créé pour automatiser la compilation du programme. Il permet de générer un exécutable nommé `programme` en tapant simplement `make` dans le terminal. Cela évite de compiler chaque fichier individuellement et simplifie le processus de développement.

## 3 Problèmes Rencontrés et Solutions

L'un des défis majeurs a été de générer toutes les images demandées en une seule exécution du programme. Pour cela, j'ai introduit le module `apply_filter`, qui contient une fonction permettant d'appliquer un filtre à une image et de sauvegarder le résultat avec un nom de fichier généré automatiquement.

**Exemple de fonction :**

```
void apply_filter(picture (*filter)(picture), picture input, char *dir,
                  char *name, char *specifier, char *ext) {
    // Appliquer le filtre
    picture result = filter(input);

    // Construire le chemin de sortie
    char *path = concat_parts(dir, name, specifier, ext);

    // Sauvegarder l'image
    write_picture(result, path);

    // Libérer les ressources
    free(path);
}
```

**Utilisation:**

```
// Conversion en couleur
apply_filter(convert_to_color_picture, gray, dir, gray_name, "convert_color", color_ext);
```

Pour les fonctions nécessitant des arguments supplémentaires, j'ai utilisé des `wrappers`. Par exemple :

```
// Éclaircir l'image en niveaux de gris
apply_filter(wrapper_brighten, gray, dir, gray_name, "brighten", gray_ext);
```

**Fonction `wrapper_brighten` :**

```
picture wrapper_brighten(picture p) {
    return brighten_picture(p, 1.5);
}
```

# 4 Conclusion

Bien que le projet réponde aux exigences demandées et génère toutes les images, il reste des améliorations possibles.

### Limites et Améliorations

### Limites

Le programme n'offre pas d'interface utilisateur, ce qui limite son utilisation. Les performances peuvent être améliorées et les algorithmes optimisés pour des images de grande taille.

### Améliorations Futures

- Ajout de nouveaux filtres avec l'utilisation de noyau gaussien (question bonus).
- Optimisation des performances pour des images de grande taille.
- Développement d'une interface utilisateur.

# 5 Annexes

## 5.1 Exemple de Commandes

`./projet Lenna_gray.pgm Lenna_color.ppm en ayant les images Lenna_gray.pgm|ppm Lenna_BW.pgm|`

## 5.2 Fichiers Soumis

- `pictures.c|.h`
- `pixels.c|.h`
- `lut.c|.h`
- `main.c`

- apply_filter.c|.h
- filename.c|.h
- Makefile
- Lenna_gray.pgm
- Lenna_color.ppm
- Lenna_BW.ppm
- inv.ppm
- Rapport PDF.
-
- Readme.md.